

Experiment 3: Creation of a Simple Network Topology Using Open-Source Network Virtualization Tools

Objective

To create and configure a simple network topology using an open-source network virtualization tool (Mininet) and verify communication between the virtual network nodes.

Tasks

- Ubuntu Linux Virtual Machine.
- VirtualBox.
- Mininet.
- Create simple Network Topology.
- Ping and basic Linux networking commands.
- After completion, stop the network.

Step 1: Ubuntu Linux Virtual Machine and VirtualBox

As set up in Experiment 1, Oracle VirtualBox was used as the hypervisor to run an Ubuntu Linux Virtual Machine. This same Ubuntu VM was used as the environment for installing and running Mininet, since Mininet requires a Linux kernel with network namespace support.

Step 2: Install and Verify Mininet

Before installation, the 'mn' command (Mininet's command-line utility) was checked and found to be unavailable, confirming that Mininet was not yet installed on the system.

```
sk-naser-hussain@sk-naser-hussain-VirtualBox: ~  
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo mn --version  
[sudo] password for sk-naser-hussain:  
sudo: mn: command not found  
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig 1. Initial check — 'sudo mn --version' returns 'command not found', confirming Mininet was not yet installed.

Mininet was then installed using the Advanced Package Tool (APT). The package index was first updated with 'sudo apt update', followed by 'sudo apt install -y mininet', which also pulled in supporting packages such as Open vSwitch and net-tools.

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo apt update  
[sudo] password for sk-naser-hussain:  
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Get:2 http://archive.ubuntu.com/ubuntu noble InRelease [126 kB]  
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
Get:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]  
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,487 kB]  
Get:6 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,263 kB]  
Get:7 http://in.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [292 kB]  
Get:8 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]  
Get:9 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1,544 kB]  
Get:10 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [324 kB]  
Get:11 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,691 kB]  
Get:12 http://in.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [339 kB]  
Get:13 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [188 kB]  
Get:14 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [12.8 kB]  
Get:15 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [508 B]  
Get:16 http://in.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [1,772 B]  
Get:17 http://in.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.6 kB]  
Get:18 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [219 kB]  
Get:19 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]  
Get:20 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1,445 kB]  
Get:21 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [336 kB]  
Get:22 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,287 kB]  
Get:23 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [242 kB]  
Get:24 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]  
Get:25 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [11.1 kB]  
Fetched 31.8 MB in 32s (948 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
218 packages can be upgraded. Run 'apt list --upgradable' to see them.  
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo apt install -y mininet  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  cgroup-tools iproute2 libcap2-bin libnftables1 libnet1-net-tools openvswitch-common openvswitch-switch python3-packaging python3-containers
```

Fig 2. Updating the package index and installing Mininet using 'sudo apt update' and 'sudo apt install -y mininet'.

After installation completed, the version check was repeated. This time, 'sudo mn --version' correctly returned '2.3.0', confirming that Mininet was installed successfully.

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo mn --version
2.3.0
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig 3. Mininet successfully installed — 'sudo mn --version' reports version 2.3.0.

Step 3: Create a Simple Network Topology

A simple network topology consisting of a single switch connected to three hosts was created using Mininet's built-in topology generator, with the command 'sudo mn --topo single,3'. Mininet created the network, added hosts h1, h2, and h3, added switch s1, added the links (h1,s1), (h2,s1), and (h3,s1), started the (OVS Bridge) switch, and dropped into the Mininet command-line interface (CLI).

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo mn --topo single,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Fig 4. Creating a single-switch, three-host topology with 'sudo mn --topo single,3', and entering the Mininet CLI.

Within the Mininet CLI, the 'nodes' command was used to list all active nodes in the topology, confirming the three hosts (h1, h2, h3) and the switch (s1).

```
mininet> nodes
available nodes are:
h1 h2 h3 s1
mininet> 
```

Fig 5. 'nodes' command listing the available nodes: h1, h2, h3, and s1.

The 'net' command was then used to display the network's links, confirming that each host's eth0 interface is connected to the corresponding port on switch s1 (h1-eth0:s1-eth1, h2-eth0:s1-eth2, h3-eth0:s1-eth3).

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0
mininet> 
```

Fig 6. 'net' command showing the links between each host and switch s1.

Step 4: Test Connectivity Using Ping and Basic Linux Networking Commands

The built-in 'pingall' command was used to test reachability between every pair of hosts in the topology. All hosts were able to reach each other, with the result showing 0% dropped (6/6 received), confirming that the topology was correctly configured and fully connected.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> █
```

Fig 7. 'pingall' command confirming full connectivity — 0% dropped (6/6 received).

The 'dump' command was used to display detailed information about each node, including the IP address and process ID assigned to each host (h1: 10.0.0.1, h2: 10.0.0.2, h3: 10.0.0.3) and the OVS bridge switch s1.

```
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=5157>
<Host h2: h2-eth0:10.0.0.2 pid=5159>
<Host h3: h3-eth0:10.0.0.3 pid=5161>
<OVSBridge s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None pid=5166>
mininet> █
```

Fig 8. 'dump' command showing IP addresses and process IDs of hosts h1, h2, h3 and switch s1.

Basic Linux networking commands were also run directly on the virtual hosts. Running 'h1 ifconfig' displayed the network interface configuration of host h1, showing its eth0 interface with IP address 10.0.0.1, along with packet statistics.

```

mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    inet6 fe80::7021:ebff:fe34:9685 prefixlen 64 scopeid 0x20<link>
    ether 72:21:eb:34:96:85 txqueuelen 1000 (Ethernet)
    RX packets 60 bytes 5577 (5.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 21 bytes 1566 (1.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet>

```

Fig 9. 'h1 ifconfig' showing host h1's network interface configuration (IP 10.0.0.1).

Finally, connectivity between two specific hosts was verified using 'h1 ping -c 4 h2', which sent 4 ICMP packets from h1 to h2. All 4 packets were transmitted and received with 0% packet loss, confirming successful host-to-host communication over the virtual network.

```

mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=10.1 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.690 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=1.18 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.088 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3092ms
rtt min/avg/max/mdev = 0.088/3.023/10.138/4.125 ms
mininet>

```

Fig 10. 'h1 ping -c 4 h2' — 4 packets transmitted, 4 received, 0% packet loss.

Step 5: Stop the Network

Once testing was complete, the 'exit' command was used at the Mininet CLI prompt to terminate the session. Mininet then automatically stopped the

controller, links, switches, and hosts that had been created for the topology.

```
mininet> exit
*** Stopping 0 controllers

*** Stopping 3 links
...
*** Stopping 1 switches
s1
*** Stopping 3 hosts
h1 h2 h3
*** Done
completed in 1121.681 seconds
```

Fig 11. 'exit' command stopping the controller, links, switches, and hosts of the single,3 topology.

As good practice, 'sudo mn -c' was also run to clean up any residual virtual interfaces, OVS datapaths, and stale Mininet processes, ensuring the system was left in a clean state for future runs.

```
sk-naser@hussaings:~$ sudo mn -c
[sudo] password for sk-naser-hussain:
*** Removing excess controllers/protocols/ovsdatapaths/pings/ovs
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflow ovs-controllerovs-testcontroller udptest mnexec lvs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflow ovs-controllerovs-testcontroller udptest mnexec lvs ryu-manager 2> /dev/null
*** Removing junk from /tmp
kill -9 -f "sudo mnexec"
*** Removing old X11 tunnels
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]*' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([_[:alnum:]]+)-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
kill -9 -f mininet
*** Shutting down stale tunnels
kill -9 -f TunnelEthernet
kill -9 -f ssh/mn
kill -9 -f ssh/mn
*** Cleanup complete.
```

Fig 12. 'sudo mn -c' cleaning up residual controllers, links, switches, and datapaths after stopping the network.

Additional Demonstration: Tree Topology (tree,3)

To further demonstrate Mininet's ability to create more complex topologies, a tree

topology was created using 'sudo mn --topo tree,3', which generated 8 hosts (h1–h8) connected through 7 switches (s1–s7) arranged in a tree hierarchy.

```
nk-naser-hussain@nk-naser-hussain-VirtualBox: $ sudo mn --topo tree,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5 h6 h7 h8
*** Adding switches:
s1 s2 s3 s4 s5 s6 s7
*** Adding links:
(s1, s2) (s1, s5) (s2, s3) (s2, s4) (s3, h1) (s3, h2) (s4, h3) (s4, h4) (s5, s6) (s5, s7) (s6, h5) (s6, h6) (s7, h7) (s7, h8)
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Starting controller

*** Starting 7 switches
s1 s2 s3 s4 s5 s6 s7 ...
*** Starting CLI:
mininet>
```

Fig 13. Creating a tree topology with 8 hosts and 7 switches using 'sudo mn --topo tree,3'.

Running 'pingall' on this larger topology confirmed full connectivity across all 8 hosts, with 0% dropped (56/56 received), demonstrating that Mininet correctly routes traffic through the multiple switch layers of the tree.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet>
```

Fig 14. 'pingall' on the tree,3 topology — 0% dropped (56/56 received).

The 'nodes' command confirmed all 8 hosts and 7 switches were active, and the

'net' command showed the multi-level link structure connecting hosts to leaf switches, and switches to each other up the tree.

```
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 h6 h7 h8 s1 s2 s3 s4 s5 s6 s7
mininet> █
```

Fig 15. 'nodes' command listing all 8 hosts and 7 switches in the tree topology.

```
mininet> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
h3 h3-eth0:s4-eth1
h4 h4-eth0:s4-eth2
h5 h5-eth0:s6-eth1
h6 h6-eth0:s6-eth2
h7 h7-eth0:s7-eth1
h8 h8-eth0:s7-eth2
s1 lo: s1-eth1:s2-eth3 s1-eth2:s5-eth3
s2 lo: s2-eth1:s3-eth3 s2-eth2:s4-eth3 s2-eth3:s1-eth1
s3 lo: s3-eth1:h1-eth0 s3-eth2:h2-eth0 s3-eth3:s2-eth1
s4 lo: s4-eth1:h3-eth0 s4-eth2:h4-eth0 s4-eth3:s2-eth2
s5 lo: s5-eth1:s6-eth3 s5-eth2:s7-eth3 s5-eth3:s1-eth2
s6 lo: s6-eth1:h5-eth0 s6-eth2:h6-eth0 s6-eth3:s5-eth1
s7 lo: s7-eth1:h7-eth0 s7-eth2:h8-eth0 s7-eth3:s5-eth2
mininet>
```

Fig 16. 'net' command showing the multi-level link structure of the tree topology.

Running 'h1 ifconfig' on this topology again confirmed host h1's interface configuration, this time showing higher packet counts due to the pingall test across all 8 hosts.

```

mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
inet6 fe80::6ce3:dbff:fe4b:298a prefixlen 64 scopeid 0x20<link>
ether 6e:e3:db:4b:29:8a txqueuelen 1000 (Ethernet)
RX packets 325 bytes 39330 (39.3 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 41 bytes 2966 (2.9 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet>

```

Fig 17. 'h1 ifconfig' on the tree topology, showing updated packet statistics after the pingall test.

Finally, the tree topology's network was stopped using 'exit' at the Mininet CLI, followed by 'sudo mn -c' to clean up all controllers, links, switches, and hosts.

```

mininet: exit
*** Stopping 0 controllers
*** Stopping 14 links
*****
*** Stopping 7 switches
s1 s2 s3 s4 s5 s6 s7
*** Stopping 8 hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Done
completed in 49226.194 seconds
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo mn -c
[sudo] password for sk-naser-hussain:
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/names
killall controller ofprotocol ofdatapath ping nox_core nox_core ovs-openflow ovs-controller ovs-testcontroller udptest minicli lvs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core nox_core ovs-openflow ovs-controller ovs-testcontroller udptest minicli lvs ryu-manager 2> /dev/null
killall -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old xll tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]*' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([a-z0-9]*)-eth[0-9]*'
ip link show
*** Killing stale mininet node processes
killall -9 -f mininet
killall -9 -f mininet
*** Shutting down stale tunnels
killall -9 -f tunnelsethernet
killall -9 -f .ssh/mn
rm -f -f .ssh/mn/*
*** Cleanup complete.
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$

```

Fig 18. Stopping and cleaning up the tree,3 topology using 'exit' followed by 'sudo mn -c'.

Result / Conclusion

Mininet was successfully installed on the Ubuntu Linux virtual machine running

under Oracle VirtualBox. A simple network topology consisting of one switch and three hosts was created and verified using Mininet's 'nodes', 'net', 'pingall', 'dump', and 'ifconfig' commands, with all hosts successfully communicating with 0% packet loss. Host-to-host connectivity was further confirmed using the ping command between two nodes. The network was then cleanly stopped using the 'exit' command and 'sudo mn -c'. A more complex tree topology with 8 hosts and 7 switches was also created and tested to demonstrate Mininet's scalability, achieving full connectivity across all nodes before being stopped and cleaned up in the same manner. This confirms that all the stated tasks of the experiment were successfully achieved.